

SIMATS ENGINEERING

CO2 ASSESSMENT TOOL 3 DEBUGGING & OPTIMISATION TASK

COURSE NAME: Design and Analysis of Algorithms

COURSE CODE: CSA0613

Name : C.Himaja

Reg.No : 192373035

Q11. Debugging Selection Sort (Incorrect Swap Position)

The given Selection Sort implementation incorrectly swaps elements even when the minimum index is unchanged, leading to unnecessary operations. Identify the logical error in

the swapping mechanism and explain its impact. Debug the code step-by-step to correct the

swapping condition. Optimize the implementation to reduce unnecessary swaps. Analyze the

time complexity of the corrected version and rewrite the optimized code with proper comments.

```
def selection_sort(arr):  
    for i in range(len(arr)):  
        min_idx = i  
        for j in range(i+1, len(arr)):  
            if arr[j] < arr[min_idx]:  
                min_idx = j  
        arr[i], arr[min_idx] = arr[min_idx], arr[i] # ISSUE  
    return arr
```

Title

Debugging and Optimization of Selection Sort Using C

Aim

To identify the logical error in the Selection Sort implementation, debug the swapping mechanism, optimize the algorithm by reducing unnecessary swaps, and analyze its performance.

Problem Statement

The given Selection Sort implementation performs swap operations even when the minimum element is already in its correct position. This leads to unnecessary operations. The objective is to identify the error, correct it, optimize the implementation, and analyze its efficiency.

Original Logic (Incorrect)

```
for(i = 0; i < n - 1; i++)
{
    minIndex = i;

    for(j = i + 1; j < n; j++)
    {
        if(arr[j] < arr[minIndex])
            minIndex = j;
    }

    // Incorrect: Swaps every time
    temp = arr[i];
    arr[i] = arr[minIndex];
    arr[minIndex] = temp;
}
```

Logical Error

The swap operation is executed every iteration, even when $\text{minIndex} == i$.

This causes the algorithm to swap an element with itself unnecessarily.

Impact of the Error

- Performs unnecessary swap operations.
- Increases execution overhead.
- Reduces efficiency.
- Does not affect the correctness of sorting.
- Makes the algorithm less optimized.

Step-by-Step Debugging

Initial Array

4500 1200 3800 2500 5100 1900

Pass 1

Current Position = 0

Minimum Element = 1200

Swap

1200 4500 3800 2500 5100 1900

Pass 2

Current Position = 1

Minimum Element = 1900

Swap

1200 1900 3800 2500 5100 4500

Pass 3

Current Position = 2

Minimum Element = 2500

Swap

1200 1900 2500 3800 5100 4500

Pass 4

Current Position = 3

Minimum Element = 3800

The incorrect program still performs

$3800 \leftrightarrow 3800$

This is an unnecessary swap.

Corrected Swapping Condition

```
if(minIndex != i)
{
    temp = arr[i];
    arr[i] = arr[minIndex];
    arr[minIndex] = temp;
}
```

Now, swapping occurs only when required.

Corrected Algorithm

1. Read the array elements.
2. Assume the current element is the minimum.
3. Search the remaining unsorted part for a smaller element.
4. Update the minimum index whenever a smaller element is found.
5. Swap only if the minimum index is different from the current index.
6. Repeat until the array becomes sorted.

Optimized Code Snippet

```
if(minIndex != i)
{
    temp = arr[i];
    arr[i] = arr[minIndex];
    arr[minIndex] = temp;
}
```

Complexity Analysis

Time Complexity

Case	Complexity
------	------------

Best Case	$O(n^2)$
-----------	----------

Average Case	$O(n^2)$
--------------	----------

Worst Case	$O(n^2)$
------------	----------

Although unnecessary swaps are removed, the number of comparisons remains the same.

Space Complexity

$O(1)$

Selection Sort is an in-place sorting algorithm and requires only constant extra memory.

Comparison

Feature	Incorrect Version	Optimized Version
Comparisons	$O(n^2)$	$O(n^2)$
Swaps	Every pass	Only when required
Efficiency	Lower	Higher
Space Complexity	$O(1)$	$O(1)$
Sorting Correctness	Correct	Correct

Advantages of the Optimized Version

- Eliminates unnecessary swaps.
- Improves execution efficiency.
- Uses constant memory.
- Easy to implement.
- Suitable for small datasets.

Disadvantages

- Still performs $O(n^2)$ comparisons.
- Not suitable for large datasets.
- Not a stable sorting algorithm.

Conclusion

The original Selection Sort algorithm sorted the array correctly but performed unnecessary self-swaps whenever the minimum element was already in its correct position. By introducing the condition `if(minIndex != i)` before swapping, unnecessary operations are eliminated. The optimized version retains the same $O(n^2)$ time complexity and $O(1)$ space complexity while improving practical performance by reducing redundant swaps.

Screenshots to Include in the Report

```

main.c
1 #include <stdio.h>
2
3 // Function to perform optimized Selection Sort
4 void selectionSort(int arr[], int n)
5 {
6     int i, j, minIndex, temp;
7
8     // Traverse the array
9     for(i = 0; i < n - 1; i++)
10    {
11        minIndex = i;
12
13        // Find the index of the minimum element
14        for(j = i + 1; j < n; j++)
15        {
16            if(arr[j] < arr[minIndex])
17            {
18                minIndex = j;
19            }
20        }
21    }
22 }
23
24 // Function to print array
25 void printArray(int arr[], int n)
26 {
27     int i;
28     for(i = 0; i < n; i++)
29         printf("%d ", arr[i]);
30 }
31
32 int main()
33 {
34     int n, arr[100];
35     printf("Enter number of transaction amounts: ");
36     scanf("%d", &n);
37     printf("Enter transaction amounts:\n");
38     for(i = 0; i < n; i++)
39     {
40         scanf("%d", &arr[i]);
41     }
42     selectionSort(arr, n);
43     printArray(arr, n);
44     return 0;
45 }

```

Output

```

Enter number of transaction amounts: 6
Enter transaction amounts:
4500
1200
3800
2500
5100
1900

Original Array:
4500 1200 3800 2500 5100 1900

Sorted Array:
1200 1900 2500 3800 4500 5100

=== Code Execution Successful ===

```

```

main.c
20 }
21
22 // Swap only if the minimum element is not already in
23 // the correct position
24 if(minIndex != i)
25 {
26     temp = arr[i];
27     arr[i] = arr[minIndex];
28     arr[minIndex] = temp;
29 }
30 }
31
32 // Function to print array
33 void printArray(int arr[], int n)
34 {
35     int i;
36     for(i = 0; i < n; i++)
37         printf("%d ", arr[i]);
38 }
39
40 int main()
41 {
42     int n, arr[100];
43     printf("Enter number of transaction amounts: ");
44     scanf("%d", &n);
45     printf("Enter transaction amounts:\n");
46     for(i = 0; i < n; i++)
47     {
48         scanf("%d", &arr[i]);
49     }
50     selectionSort(arr, n);
51     printArray(arr, n);
52     return 0;
53 }

```

Output

```

Enter number of transaction amounts: 6
Enter transaction amounts:
4500
1200
3800
2500
5100
1900

Original Array:
4500 1200 3800 2500 5100 1900

Sorted Array:
1200 1900 2500 3800 4500 5100

=== Code Execution Successful ===

```

main.c

Run

Clear

38 }

39

40 int main()

41 {

42 int n, i;

43

44 printf("Enter number of transaction amounts: ");

45 scanf("%d", &n);

46

47 int arr[n];

48

49 printf("Enter transaction amounts:\n");

50

51 for(i = 0; i < n; i++)

52 scanf("%d", &arr[i]);

53

54 printf("\nOriginal Array:\n");

55 printArray(arr, n);

Enter number of transaction amounts: 6

Enter transaction amounts:

4500

1200

3800

2500

5100

1900

Original Array:

4500 1200 3800 2500 5100 1900

Sorted Array:

1200 1900 2500 3800 4500 5100

=== Code Execution Successful ===

main.c

Run

Clear

45 scanf("%d", &n);

46

47 int arr[n];

48

49 printf("Enter transaction amounts:\n");

50

51 for(i = 0; i < n; i++)

52 scanf("%d", &arr[i]);

53

54 printf("\nOriginal Array:\n");

55 printArray(arr, n);

56

57 selectionSort(arr, n);

58

59 printf("\n\nSorted Array:\n");

60 printArray(arr, n);

61

62 return 0;

63 }

Enter number of transaction amounts: 6

Enter transaction amounts:

4500

1200

3800

2500

5100

1900

Original Array:

4500 1200 3800 2500 5100 1900

Sorted Array:

1200 1900 2500 3800 4500 5100

=== Code Execution Successful ===